

SFC-RG JOINT LECTURE

Threat Clarification and Correctness Proofs in Protocol Design

Graduate School of Media and Governance, Keio University

Riku MOCHIZUKI

moz@sfc.keio.ac.jp

May 1, 2025

Topics and Objectives

Topics

- Introduction to protocol design
- Introduction to threat models
- Introduction to formal proofs of protocol correctness
- Introduction to research challenges in distributed systems and digital identity

Objectives

- Understand the principles of protocol design in distributed systems
- Understand the approach to correctness guarantees in protocol design
- Understand how to structure academic papers about protocols and what aspects need to be clarified

What is a Protocol?

Definition of Protocol (Oxford Dictionary)

A set of rules governing the exchange or transmission of data between entities.

Entity: Refers to individual participants (users, devices) in a protocol.

Definition of Algorithm (Oxford Dictionary)

A set of instructions for performing a task or solving a problem.

While algorithms focus on computational procedures, protocols emphasize interactions between participants. ¹ Common protocol examples: HTTP, SMTP, TLS/SSL, SSH, OAuth, Paxos, Raft

¹Protocols are often explained as conversing using common rules like English or Japanese, which represents interaction rules. The term “Japanese algorithm” is not common because algorithms represent computational procedures rather than interactions.

How to Design Correct Protocols

This lecture does not cover protocol design methodologies.

This lecture focuses on how to explain and demonstrate the correctness of protocols you design in your domain

To demonstrate protocol correctness, the following aspects must be clarified:

- Which entities participate in the protocol
- What information exchanges occur between entities
- What conditions must be satisfied for the protocol to be achieved the goal
- **What behaviors each entity might exhibit**
- **How to determine whether the protocol is correct² given these entity behaviors**

²Satisfying both Safety (bad things don't happen) and Liveness (good things eventually happen) properties

Adversaries and Threat Models

In protocol design, clearly defining assumptions about entity capabilities and behaviors is crucial when considering what behaviors to allow.

Adversary (Oxford Dictionary)

An adversary refers to an attacker, typically with malicious intent, who undertakes actions against a system or protocol

In protocol design, an adversary can control entities participating in the protocol, modifying or monitoring their operations.

Threat Model

A threat model formalizes assumptions about adversary capabilities and behaviors.

Adversaries and Threat Models

Adversaries control honest entities and introduce corrupted entities into the protocol.

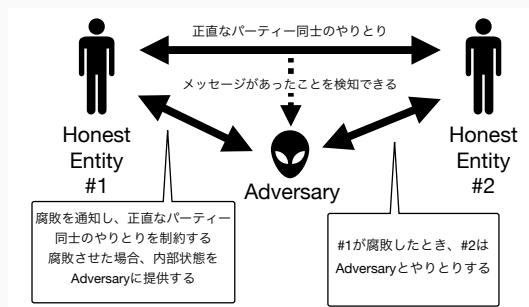


Figure 1: Relationship between adversaries and entities. Adversaries can introduce corrupted entities into the protocol by intercepting messages from honest entities or directly controlling entities. (Note that entities do not become corrupted directly!)

Components of Threat Models

Threat models consist of 5 key components:

1. **Corruption Model.** Behavior patterns of entities controlled by the adversary
2. **Computational Power.** Constraints on computational resources available to the adversary
3. **Adaptivity.** Timing and methods by which the adversary corrupts entities
4. **Visibility.** Range of information observable by the adversary
5. **Mobility.** Patterns of entity corruption and recovery by the adversary

Different combinations of these components define different threat models. The selection of components depends on the protocol's purpose and the environments in which it may be executed.

Corruption Models

Corruption models define how adversaries can control entities.

- **Passive.** Follows the protocol completely while accumulating and utilizing all observed information. Cannot deviate from protocol specifications.³⁴
- **Crash.** Can stop responding at any time but operates according to the protocol outside of downtime.
- **Omission.** Entities can arbitrarily delete or ignore received messages but cannot modify the content of messages they send.
- **Byzantine.** Entities can arbitrarily deviate from protocol specifications.

Each model encompasses the previous models: Byzantine $\supset$ Omission $\supset$ Crash $\supset$ Passive

³For example, a bank employee who correctly processes customer transaction data while secretly recording and using that data for personal purposes.

⁴Also known as Honest-but-Curious or Semi-Honest model.

Adaptivity Models

Adaptivity models define when and how adversaries corrupt participants:

- **Static:** Adversaries select entities to corrupt before protocol execution and cannot change their selection during execution.
- **Adaptive:** Adversaries can dynamically select entities to corrupt during protocol execution.

Consider the Adaptive model when there are few entities, many communication phases in the protocol, or long protocol execution times. For example, key exchange models typically consider the Static model. Consensus algorithms typically consider the Adaptive model.

Visibility Models

Visibility models define the extent to which adversaries can observe messages and internal states.

- **Full Information.** Adversaries can completely observe all message contents and internal states (execution variables) of each entity
- **Private Channels.** Communication content between honest entities is completely secret; adversaries can know that messages are being sent and received but cannot see their contents. Internal states of honest entities are also hidden from adversaries.
- **Erasure Model.** Extends Private Channels by adding the constraint that adversaries cannot observe or recover data erased before an entity becomes corrupted.
- **Oblivious.** Message header information (sender, recipient, length) is observable by adversaries, but message content (payload) is completely hidden. The existence of communication and its metadata are exposed, but the content is protected.⁵

⁵This concept is used in specific encrypted communication protocols and anonymous communication systems.

Computational Power Models

Computational power models define limitations on adversaries' computational resources:

- **Unbounded.** Adversaries with unlimited computational resources. Theoretically capable of breaking all cryptography.
- **Polynomially Bounded.** Adversaries limited to polynomial-time computation⁶
- **Fine-grained.** Adversaries with specific computational limitations.
For example, models with limitations on hash computations per second (Hashrate)

The focus is on the adversary's computational power, not the entities' computational power. Adversaries can observe entity messages and internal states according to the Visibility Model. This model defines what computational resources adversaries can use to utilize that information.

⁶When solving a problem, the computational complexity can be expressed as order $O(n^c)$ where $n, c \in \mathbb{Z}$. Examples include the difficulty of discrete logarithm problems and integer factorization problems. Generally, Probabilistic Polynomial Time (PPT) refers to algorithms that use randomness and always terminate within polynomial time λ^c relative to the security parameter $\lambda \in \mathbb{Z}$ (e.g., key length)

Mobility Models

Mobility models define patterns of entity corruption and recovery by adversaries:

- **Fixed.** Once corrupted, entities remain corrupted throughout protocol execution and cannot recover
- **Mobile.** Adversaries can dynamically corrupt and recover entities. However, there is an upper limit on the total number of corrupted entities at any given time⁷

From the perspective of IoT, mobile devices, and long-term system operation, the Mobile model is generally considered.

⁷Be careful not to confuse this with the Adaptive model in Adaptivity Models. Adaptive refers to the ability of adversaries to choose when to corrupt entities. Mobility considers whether corrupted entities can recover and become honest entities again under those conditions.

Importance of Appropriate Assumptions

In security protocol research, clarifying assumptions is extremely important.

- If assumptions are too weak: Protocols may be vulnerable to realistic attacks
- If assumptions are too strong: Design and implementation may become complex, potentially reducing performance and practicality

Value of explicit assumptions:

- Enables accurate understanding of protocol correctness
- Allows fair comparison of different protocols using the same criteria

In actual system design, selecting appropriate assumptions (threat models) based on specific uses and potential risks is important.

Example: Practical Byzantine Fault Tolerance (PBFT)

PBFT

- PBFT is a replication protocol that **tolerates Byzantine faults** while achieving linearizability⁸⁹
- This holds only when $n \geq 3f + 1$ (where f is the number of adversaries and n is the total number of replicas).

The PBFT protocol was the first to demonstrate achieving linearizability in the partial synchrony model. Even 20 years after its introduction, PBFT remains widely used.

⁸Linearizable replication means all replicas process client requests in the same order, and changes are always reflected within the write request session. Each replicated data is assigned a sequence number and processed according to this sequence number.

⁹In Turing-complete state machines, replicated data is viewed as operations (state transition functions). When operations f and g exist, generally $f \circ g \neq g \circ f$, so execution order is important.

PBFT Protocol Details (1)

Main phases of PBFT:

1. **Pre-prepare:** Primary (leader) p assigns sequence number s to client request r and sends $\langle \text{PRE-PREPARE}, s, r \rangle$ message to other replicas
2. **Prepare:** Replicas agree on request order and send $\langle \text{PREPARE}, s, r \rangle$ message to all replicas
3. **Commit:** Replicas that receive at least $2f + 1$ PREPARE messages send $\langle \text{COMMIT}, s, r \rangle$ message, committing to execution
4. **Reply:** Replicas that receive $2f + 1$ COMMIT messages execute the request and send results to the client

PBFT Protocol Details (2)

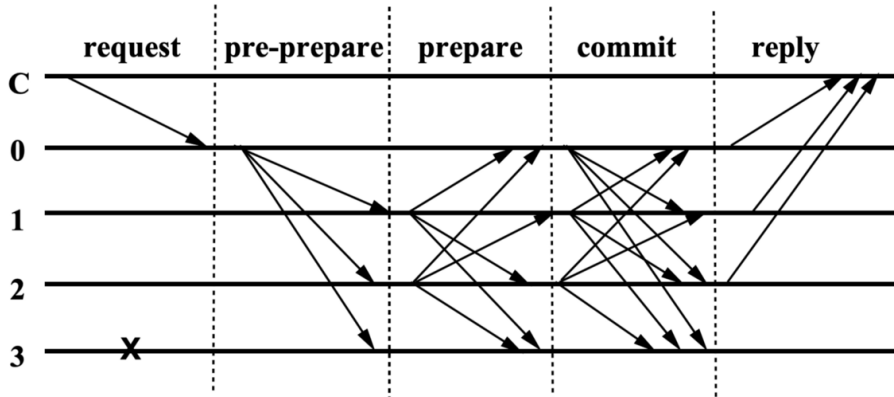


Figure 2: PBFT sequential diagram

PBFT Threat Model

Let's consider the PBFT threat model.

PBFT Threat Model Λ_{PBFT}

- **Corruption:** Byzantine - Replicas can behave arbitrarily, such as sending different messages to different replicas
- **Adaptivity:** Adaptive - Adversaries can dynamically corrupt different replicas in each phase of the protocol
- **Computational Power:** Polynomially Bounded - Limited to polynomial-time computation
- **Visibility:** Private channels - Communication content between honest replicas is secret, and internal states are not visible to adversaries
- **Mobility:** Fixed - Once corrupted, replicas remain corrupted throughout protocol execution and cannot recover

PBFT Correctness Guarantees

PBFT Correctness Guarantees

When $n \geq 3f + 1$ (where f is the number of corrupted nodes and n is the total number of replicas):

- **Safety**: Honest replicas never commit different values for the same sequence number
- **Liveness**: All client requests are eventually processed and responses are returned

If we can prove that PBFT satisfies both Safety and Liveness, then PBFT can be considered correct.

In other words, if we can prove that PBFT satisfies Safety and Liveness regardless of what attacks adversaries perform within the threat model Λ_{PBFT} , then PBFT can be considered correct.

Question: Why is Safety satisfied when $n \geq 3f + 1$? Let's explain why PBFT is secure!

PBFT Safety Proof

Assumption: Two different values v and v' are committed with the same sequence number s .

Lemma

For each value to be committed, COMMIT messages from at least $2f + 1$ replicas are required. (Out of $3f + 1$ total replicas, at most f can be corrupted)

- Value v requires COMMIT messages from at least $2f + 1$ replicas in set $\mathcal{P}$
- Similarly, value v' requires COMMIT messages from at least $2f + 1$ replicas in set $\mathcal{Q}$
- $|\mathcal{P} \cap \mathcal{Q}| \geq 2f + 1 + 2f + 1 - n \geq 4f + 2 - (3f + 1) = f + 1$
- The intersection $\mathcal{P} \cap \mathcal{Q}$ contains at least $f + 1$ replicas, and since the system has at most f Byzantine replicas,
at least 1 honest replica must have sent COMMIT messages for both values v and v'
- However, honest replicas never send COMMIT messages for different values with the same sequence number, creating a contradiction
- Therefore, the assumption is false

Correctness Proofs for Complex Protocols

In practice, simple protocols like PBFT are rare, and composite protocols that combine existing protocols as components are common.

- Application layer protocols are often built on existing replication layers (e.g., PBFT) ¹⁰

Hybrid Protocol

A hybrid protocol obtained by combining n protocols $\pi_1, \dots, \pi_n$ is denoted as $\pi[\pi_1, \dots, \pi_n]$.

Motivation: How to demonstrate the correctness of composite protocol $\pi[\pi_1, \dots, \pi_n]$?

- Can we analyze each component individually to prove the correctness of the whole?
- How to guarantee correctness in any environment?

¹⁰Example: Building a timestamp mechanism on top of PBFT. Related to kekeho's GP2.

Universal Composability Framework

Using the UC Framework, the correctness of complex protocols can be demonstrated based on the completeness guarantees of simple components. In other words, to prove that a composite protocol $\pi[\pi_1, \dots, \pi_n]$ is correct, we need to prove the correctness of each component $\pi_1, \dots, \pi_n$ and show that they are combined appropriately.

Definition of Ideal Functionality $\mathcal{F}$

An ideal functionality $\mathcal{F}$ receives inputs from entities, computes f , and returns outputs to each entity. In other words, it defines the protocol interface and defines a mapping f that returns outputs for any input.¹¹

Definition of Protocol π

A protocol π is an actual program, a procedure executed between multiple entities (concrete implementation of the protocol). It is designed to realize the ideal functionality $\mathcal{F}$.

¹¹This can be considered as an indestructible (i.e., the interface or mapping f cannot be modified) trusted entity.

UC has two worlds for executing ideal functionality $\mathcal{F}$ and protocol π :

Ideal World

- The ideal functionality $\mathcal{F}$ performs computations, and the adversary behaves as a simulator Sim .
- Dummy entities send inputs to $\mathcal{F}$ and return outputs from $\mathcal{F}$ to the environment $\mathcal{Z}$.
- The simulator Sim mimics the behavior of the real-world adversary through the interface with the ideal functionality $\mathcal{F}$ and returns the results to the environment.

Ideal/Real World Paradigm

UC has two worlds for executing ideal functionality $\mathcal{F}$ and protocol π :

Real World

- The real world consists of the actual protocol π , its participating entities, and the adversary $\mathcal{A}$.
- Entities communicate with each other according to protocol π while advancing the computation.
- The adversary $\mathcal{A}$ controls corrupted entities, and all internal states and actions are controlled by $\mathcal{A}$.

In other words... each world can be interpreted as follows:

- The ideal world can be considered as "a world where a trusted entity $\mathcal{F}$ performs all processing."
- The real world can be considered as "a world where entities communicate with each other to perform processing because there are no trusted entities."
- Therefore, a protocol being correct means that the protocol π executed by entities in the real world is indistinguishable from the ideal functionality $\mathcal{F}$ executed in the ideal world.^a

^aMore precisely, it is indistinguishable with negligible probability. This is because UC holds only against probabilistic polynomial-time (PPT) adversaries. Regarding probabilistic polynomial time, as mentioned earlier, it can be approximated as order $O(n^c)$ in a probabilistic Turing machine.

Environment Entity Operating Ideal and Real Worlds

The entity that operates these two worlds is the environment $\mathcal{Z}$.

Environment $\mathcal{Z}$

- The environment $\mathcal{Z}$ abstracts all systems outside the protocol (such as other protocols).
- In the ideal world, $\mathcal{Z}$ sends protocol inputs to dummy entities and receives their outputs and outputs from the simulator Sim .¹²
- In the real world, $\mathcal{Z}$ sends inputs to entities executing the protocol and receives their outputs and outputs from the adversary $\mathcal{A}$.
- $\mathcal{Z}$ tries to distinguish whether it is interacting with the ideal world or the real world.¹³

¹²More precisely, the environment does not communicate directly with $\mathcal{F}$ but interacts indirectly through dummy entities

¹³In other words, $\mathcal{Z}$ tries to find differences between the behavior of the actual protocol and entities in the real world and the behavior of the ideal functionality and dummy entities in the ideal world.

Comparison of Ideal and Real Worlds

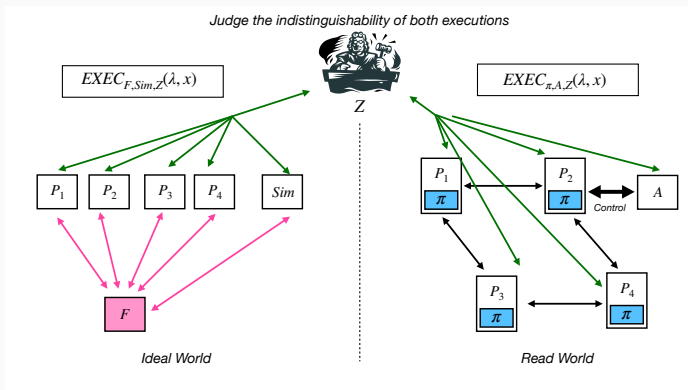


Figure 3: Interaction between ideal and real worlds. In the ideal world, each dummy party P_i communicates directly with the ideal functionality $\mathcal{F}$, while in the real world, entities P_i executing protocol π communicate with each other. This perspective allows $\mathcal{F}$ to be considered as a trusted ideal functionality.

Correctness Claims in UC

Definition of UC Correctness

If the environment $\mathcal{Z}$ tries to find differences between interactions in the real world and the ideal world but cannot distinguish between them, then π can be said to correctly realize the ideal functionality $\mathcal{F}$. This is called “ π UC-realizes $\mathcal{F}$.”

Rigorous Definition. For security parameter $\lambda \in \mathbb{N}$ and protocol π , we say that π “UC-realizes” the ideal functionality $\mathcal{F}$ when: For any probabilistic polynomial-time¹⁴ adversary $\mathcal{A}$, there exists a probabilistic polynomial-time algorithm Sim such that for any probabilistic polynomial-time environment $\mathcal{Z}$ and any input $x \in \{0, 1\}^*$:

$$EXEC_{\mathcal{F}, Sim, \mathcal{Z}}(\lambda, x) \approx EXEC_{\pi, \mathcal{A}, \mathcal{Z}}(\lambda, x)$$

¹⁴UC holds only against probabilistic polynomial-time adversaries

Benefits of Using UC for Correctness Proofs

- **Modularity.** Complex tasks can be decomposed into smaller protocols, and the correctness of each protocol can be proven independently.
- **Composition Theorem.** If protocol π UC-realizes ideal functionality $\mathcal{F}$ and protocol ρ using $\mathcal{F}$ UC-realizes ideal functionality $\mathcal{G}$, then the protocol $\rho^{\mathcal{F} \rightarrow \pi}$ obtained by replacing $\mathcal{F}$ with π also UC-realizes $\mathcal{G}$.

The core of the UC framework is "composability": the correctness of a protocol, once proven, is maintained in any complex environment. This enables the incremental construction and reuse of secure protocols.

Protocol Instances and Session Identification

In the real world, multiple copies of the same protocol are executed simultaneously by different users. A protocol is like a type, and instances of that type are actually executed.

Protocol Instance

An instance of protocol π is a specific execution of that protocol, typically distinguished by a unique identifier.

- **Session Identifier** (sid): A unique identifier that distinguishes each protocol execution
- For example... on blockchains, smart contract addresses function as sid

In the basic UC framework, each protocol instance has its own instance. However, this may not be realistic in some cases (e.g., when the same hash function is used in multiple protocols).

Generalized Universal Composability (GUC) Framework

What is the GUC Framework

An extension of the UC framework that introduces the concept of "global functionalities" shared across multiple protocol instances.

Global Functionality

- Functionality shared across multiple protocol instances
- Examples: hash functions, digital signatures, shared ledgers
- Maintains consistent internal state across all protocol sessions

GUC not only enables parallel execution environments but also eliminates the need to analyze instances individually.

Why Learn These Correctness Proof Methods?

To write academic papers in formats accepted by the academic community.^a Without these methods, papers may be rejected outright even if the protocol design appears good.

^aEspecially for international journals and top conferences. Top conferences have acceptance rates of 20% or lower, or CORE rank A or A⁺ conferences

Simply describing protocol specifications and implementations is insufficient

“ Since there is no formal security model, it is hard to verify the formal correctness of the claims. Moreover, the scheme includes many low-level implementation details instead of giving a general description of their scheme. ”

(Excerpt from a review comment received by moz two years ago from CRYPTO'23, a top conference)



International Association for Cryptologic Research (IACR)

<https://eprint.iacr.org> › ... PDF

頻繁にアクセス

A New Paradigm for Cryptographic Protocols

R. Canetti 著 · 2000 · 被引用数: 4519 — **Composability** (or, rather, **de-composability**) can be regained

Let's Examine a Paper Published in a Top Conference

Let's examine a paper titled "FairSwap: How to Fairly Exchange Digital Goods" published in ACM CCS 2020, a top conference.

This paper proposes a protocol for exchanging digital data x and y between $\mathcal{A}$ and $\mathcal{B}$. The protocol guarantees that either both conditions $P : \mathcal{A} \xrightarrow{x} \mathcal{B}$ and $Q : \mathcal{B} \xrightarrow{y} \mathcal{A}$ are satisfied, or neither is satisfied.

The paper defines the ideal functionality $\mathcal{F}_{fe}$ along with the global functionality $\mathcal{L}$ with blockchain capabilities and a random oracle $\mathcal{H}$.

<https://dl.acm.org/doi/pdf/10.1145/3243734.3243857>



FairSwap: How to Fairly Exchange Digital Goods

Stefan Dziembowski
Institute of Informatics
University of Warsaw, Poland
stefan.dziembowski@crypto.edu.pl

Lisa ECKEY
Department of Computer Science
TU Darmstadt, Germany
lisa.eckey@cs.tu-darmstadt.de

Sebastian Faust
Department of Computer Science
TU Darmstadt, Germany
sebastian.faust@cs.tu-darmstadt.de

ABSTRACT

We introduce FairSwap – an efficient protocol for fair exchange

Unfortunately, it has been shown that without further assumptions it is impossible to design protocols that guarantee such strong fairness guarantees [90]. We propose to circumvent this impossibility

Are These Assumptions Sufficient?

The assumptions we have used so far are:

- **Network Model:** Asynchronous, synchronous, or partially synchronous (with an upper bound on communication delay)
- **Adversary Model:** How adversaries behave
- **Number of Corrupted Nodes:** Conditions on the number of entities the attacker can control (e.g., $n \geq 3f + 1$)

Question: Are these the only threats and assumptions to consider when operating in the real world?

Sybil Attack

With just these assumptions, all protocols are vulnerable to Sybil Attacks.

Sybil Attack

An attack where a single entity impersonates multiple entities to gain undue influence in a system.¹⁵

PBFT requires the condition $n \geq 3f + 1$, where n is the number of nodes and f is the number of adversaries. This implicitly assumes that each replica is uniquely identifiable and message origins are authentic. If an adversary can behave as multiple entities, this condition breaks down.^a

^aThis allows certain compromises in the Byzantine Corruption Model. Is this acceptable?

¹⁵One entity plays the roles of two different identities, but other entities cannot determine whether these identities belong to the same entity.

Digital Signatures

To verify the authenticity of message origins, messages must be digitally signed and these signatures must be verified.

Digital Signatures

Digital signatures use mathematically related public key (verification key) pk and private key (signing key) sk pairs to verify message origins.

- Signing a message m : $\sigma \leftarrow \text{SIGN}(sk, m)$
- Verifying a message m with signature σ : $\{0, 1\} \leftarrow \text{VERIFY}(pk, m, \sigma)$ where 1 if m is valid, 0 otherwise

Specific digital signature schemes include RSA, ECDSA, EdDSA, and Schnorr signatures.

Verifying Message Authenticity

When entity $\mathcal{P}$ sends a message m to entity $\mathcal{Q}$, entity $\mathcal{Q}$ needs to verify that message m was indeed received from $\mathcal{P}$.

- Entity $\mathcal{P}$ generates a digital signature σ for message m using the private key $sk_{\mathcal{P}}$ corresponding to public key $pk_{\mathcal{P}}$.
- $\mathcal{P}$ sends (m, σ) to $\mathcal{Q}$.
- $\mathcal{Q}$ verifies the received (m, σ) : $\text{VERIFY}(pk_{\mathcal{P}}, m, \sigma) \stackrel{?}{=} 1$

Assumption needed to ensure verification validity: $\mathcal{Q}$ knows $\mathcal{P}$'s public key $pk_{\mathcal{P}}$.

Question: Is this sufficient to prevent Sybil Attacks?

Cost of Generating Identifiers

Answer: Insufficient. The proposition “Digital signatures can distinguish message origins $\Leftrightarrow$ Entities can be distinguished” does not necessarily hold.

Digital signatures can distinguish message senders, but verification validity is not guaranteed unless a one-to-one correspondence between entities and public keys is assumed.

- A single entity can generate multiple public keys and behave as multiple entities
- A Key Management System¹⁶ is necessary to ensure one-to-one correspondence between entities and public keys. KMS verifies this correspondence¹⁷

¹⁶Key Management System (KMS) guarantees one-to-one correspondence between entities and public keys. Public Key Infrastructure (PKI) is a type of Key Management System.

¹⁷Verification involves two aspects. First, verifying that an entity claiming to be a specific identity is indeed that identity. Second, defining a mapping between entities and public keys, and confirming this relationship is bijective. After confirmation, KMS issues a signature (=certificate) indicating the binding is valid. Users can substitute this verification with signature verification by trusting the KMS.

Problems with KMS

However, KMS has the following issues:

- Allowing certification authorities = assuming the existence of trusted third parties
- Creates single points of failure and scalability problems

Other approaches like **Proof-of-Work** require computational resources for creating new identifiers.

- When examining protocols (with Byzantine Fault Tolerance) in the context of distributed systems, bottlenecks exist (implicit, non-trivial assumptions) that prevent maximizing the appeal and effectiveness of distributed systems. These are not always documented in academic papers.

One more thing

How to find impactful problems?

- Having operational experience expands the options for interpreting protocols discussed in theory.
- This means changing the situation in which a protocol operates (assumptions supporting the logic of the proposed protocol^a) based on insights gained from operations, and determining whether bottlenecks can be ignored or not.
- I believe such exploration contributes to impactful and clear problem statements.

^aFor example, adversary models, network, and timing models

This lecture provided a theoretical perspective, but hearing operational viewpoints from other kGs may bring new interpretations to your research (field). I hope this general lecture becomes a forum where we organically generate, share, and enhance such interpretations together.

SFC-RG 全体講義

プロトコル設計における脅威の明確化と正当性証明

慶應義塾大学 大学院政策・メディア研究科

Riku Mochizuki

moz@sfc.keio.ac.jp

May 1, 2025

主題

- ・ プロトコルの設計について紹介する
- ・ 脅威モデルについて紹介する
- ・ プロトコルの正当性証明 (Formal Proof) について紹介する
- ・ 最後に Delight 的な (分散システムとデジタルアイデンティティについての) 研究課題を紹介する

目的

- ・ 分散システムにおけるプロトコルの設計 (のお気持ち) を理解する
- ・ プロトコルの設計における正当性保証の考え方 (のお気持ち) を理解する
- ・ 自身のプロトコルを学術論文としてまとめる際、どのような構成で何を明らかにすれば良いかを理解する

プロトコルとは？

プロトコルの定義 (Oxford Dictionary)

A set of rules governing the exchange or transmission of data between entities.

エンティティ：プロトコルに参加する個々の主体（ユーザー、デバイス）を指す。

アルゴリズムの定義 (Oxford Dictionary)

A set of instructions for performing a task or solving a problem.

アルゴリズムは計算手順に焦点を当てるのに対し、プロトコルは参加者間の相互作用に重点を置く。¹一般的なプロトコルの例：HTTP、SMTP、TLS/SSL、SSH、OAuth、Paxos、Raft

¹プロトコルは英語や日本語など共通のルールで会話する例として説明されることが多いが、これは相互作用の規則を表しているためである。一方で「日本語アルゴリズム」という表現が一般的でないのは、アルゴリズムが相互作用ではなく計算手順を表すためである。

正当 (Correctness) なプロトコルの設計をどうやるか

この講義はプロトコルの設計手法については紹介しない。

皆さんが自分のドメインで設計したプロトコルがなぜ正当 (Correctness) と言えるのかをどのように他人に説明できるかを考える講義

プロトコルの正当性を示すには、以下の点を明確にすることが重要である：

- ・ どのようなエンティティがプロトコルに関与するか
- ・ エンティティ間でどのような情報交換が行われるか
- ・ プロトコルはどのような仕様を満たせばよいのか
- ・ それぞれのエンティティがどのような振る舞いをする可能性があるか
- ・ そのようなエンティティが存在するときに、プロトコルが正当²であるかどうかをどのように判断するか

²Safety (悪いことが起こらない) , Liveness (良いことがいつか起こる) のプロパティを共に満たすか

敵対者と脅威モデル

プロトコル設計において、どのような振る舞いを許容するかを考える上では、エンティティの能力と行動に関する仮定を明確にすることが重要である。

敵対者の定義 (Oxford Dictionary)

An adversary refers to an attacker, typically with malicious intent, who undertakes actions against a system or protocol

プロトコル設計においては、敵対者は、プロトコルに参加するエンティティを制御下に置き、そのエンティティの動作を変更または監視できる状態。

脅威モデル

脅威モデル (threat model) とは、敵対者の能力と行動に関する仮定を形式化したモデル。

敵対者と脅威モデル

敵対者は、誠実なエンティティを制御し、プロトコルに腐敗したエンティティを導入する。

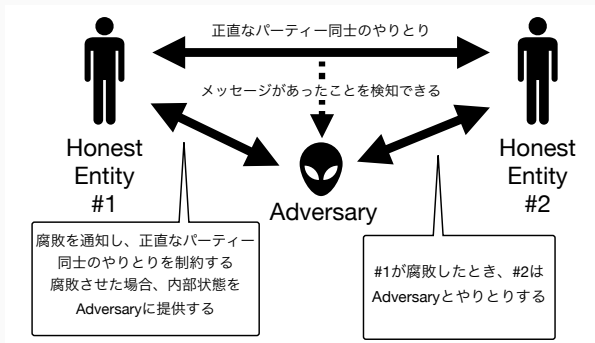


Figure 1: 敵対者とエンティティの関係。敵対者は誠実なエンティティのメッセージを傍受したり、直接エンティティを制御したりすることで、プロトコルに腐敗したエンティティを導入することができる。(エンティティが直接、腐敗することはないことに注意！)

脅威モデルの構成要素

脅威モデルは以下の5つの要素から構成される：

1. Corruption Model. 敵対者が制御するエンティティの行動様式
2. Computational Power. 敵対者が利用できる計算リソースの制約
3. Adaptivity. 敵対者がエンティティを腐敗させるタイミングと方法
4. Visibility. 敵対者が観察できる情報の範囲
5. Mobility. 敵対者によるエンティティの腐敗と回復のパターン

この要素の組み合わせ方によって、異なる脅威モデルが定義される。どのような要素を選択するかは、プロトコルの目的、実行されうる環境によって決まる。

腐敗モデル (Corruption Models)

腐敗モデルは、敵対者はエンティティをどのように制御できるかを定義する。

- ・ Passive. プロトコルに完全に従いながら、観測したすべての情報を蓄積して活用できる。しかし、プロトコルの仕様に逸脱することはできない。³⁴
- ・ Crash. 応答を任意のタイミングで停止できるが、停止時間外は正確にプロトコルに従って動作する。
- ・ Omission. エンティティは受信したのメッセージを任意に削除または無視できる。送信するメッセージの内容を変更することはできない。
- ・ Byzantine. エンティティはプロトコル仕様から任意に逸脱できる。

各モデルは前のモデルを包含している：Byzantine $\supset$ Omission $\supset$ Crash $\supset$ Passive

³具体例として銀行の従業員、顧客の取引データを正しく処理しながらも、そのデータをこっそり記録して個人的に利用する場合。

⁴Honest-but-Curious, Semi-Honest モデルとも呼ばれる。

適応性モデル (Adaptivity Models)

適応性モデルは、敵対者がいつ、どのように参加者を腐敗させるかを定義する：

- ・ Static：敵対者は腐敗させるエンティティをプロトコル実行前に選ぶ。実行中に変更できない。
- ・ Adaptive：敵対者はプロトコル実行中に腐敗させるエンティティを動的に選択できる。

エンティティが少ない場合、プロトコルの通信フェーズ数が多い場合、プロトコルの実行時間が長い場合、Adaptive モデルを考慮する。例えば、鍵交換モデルなどでは、Static モデルを考えることが一般的。コンセンサスアルゴリズムでは、Adaptive モデルを考えることが一般的。

可視性モデル (Visibility Models)

可視性モデルは、敵対者がどの程度のメッセージや内部状態を観察できるかを定義する。

- ・ Full Information. 敵対者はすべてのメッセージ内容と各エンティティの内部状態（実行中の変数）を完全に観察可能
- ・ Private Channels. 誠実なエンティティ間の通信内容は完全に秘密であり、敵対者はメッセージの送受信の事実を知ることにはできても内容を見ることはできない。また誠実なエンティティ内部状態も敵対者からは隠蔽される。
- ・ Erasure Model. Private Channels に対して、参加者が腐敗する前に消去されたデータは、敵対者は観察・復元できない制約を足したもの。
- ・ Oblivious. メッセージのヘッダー情報（送信元、宛先、長さ）は敵対者に観察可能だが、メッセージの内容（ペイロード）は完全に隠蔽される。通信の存在自体とそのメタデータは露出するが、内容は保護される。⁵

暗号プロトコルでは、機密性 (intended to be kept secret) を保証するために、Private Channels モデルを考えることが多い。

⁵特定の暗号化通信プロトコルや匿名通信システムで利用される概念である。

計算能力モデル (Computational Power Models)

計算能力モデルは、敵対者の計算資源の制限を定義する：

- ・ Unbounded. 無制限の計算資源を持つ敵対者。理論上はすべての暗号を破ることができる。
- ・ Polynomially Bounded. 多項式時間の計算⁶に制限される敵対者
- ・ Fine-grained. 特定の計算制限を持つ敵対者。
例えば、秒間ハッシュ計算回数に制限があるモデル (Hashrate)

重要なのは、エンティティの計算能力ではなく、敵対者の計算能力である。敵対者は Visibility Models にそって、エンティティのメッセージや内部状態を観察できる。その情報を敵対者はどのような計算資源を用いて活用できるかを定義する。

⁶ある問題 (Problem) を解くときに、その問題の解を求める計算複雑性がオーダー $O(n^c)$ $n, c \in \mathbb{Z}$ で表せる。
例えば、離散対数問題、素因数分解問題の難しさなど。

一般的には Probabilistic Polynomial Time (PPT) が確率チューリングマシンにおいて、入力となるセキュリティパラメータ $\lambda \in \mathbb{Z}$ (鍵長など) の大きさに対して、乱数を用いながらも 多項式時間 λ^c 以内に必ず終了するアルゴリズム

移動性モデル (Mobility Models)

移動性モデルは、敵対者による参加者の腐敗と回復のパターンを定義する：

- ・ Fixed. 一度腐敗したエンティティはプロトコル実行中ずっと腐敗したままであり、回復することはない
- ・ Mobile. 敵対者はエンティティを動的に腐敗させたり、回復させたりできる。ただし、任意の時点で腐敗しているエンティティの総数には上限がある⁷

IoT やモバイルデバイス、システムの長期運用の視点を考えると、一般的に Mobile モデルを考える。

⁷Adaptivity Models の Adaptive との混同に注意すること。Adaptive は敵対者がエンティティを腐敗させるタイミングを選択できることを指す。Mobility はその条件下において、一度腐敗させたエンティティが正常なエンティティとして回復するかどうかを考慮している。

適切な仮定を置くことの重要性

セキュリティプロトコルの研究において、**仮定の明確化**は極めて重要である。

- ・ 仮定が弱すぎる場合. 現実的な攻撃に対して脆弱なプロトコルになる可能性がある
- ・ 仮定が強すぎる場合. 設計や実装が複雑になり、その影響でパフォーマンスが下がり、実用性が低下する可能性がある

仮定を明示する価値：

- ・ **プロトコルの正当性**を正確に理解できる
- ・ 異なるプロトコルを同じ基準で公正に比較できる

実際のシステム設計では、特定の用途や発生しうるリスクに基づいて適切な仮定（脅威モデル）を選択することが重要である。

例：Practical Byzantine Fault Tolerance (PBFT)

PBFT

- ・ PBFT は、Byzantine 障害を許容しつつも、Linearizable⁸を達成するレプリケーションプロトコル⁹である。
- ・ ただし、 $n \geq 3f + 1$ （ここで f は敵対者の数、 n はレプリカの総数）のときに限る。

PBFT プロトコルは、部分同期モデルにおいて初めて Linearizable を達成することを示したプロトコルである。発表から 20 年経った現在でも、PBFT は依然として広く使われている。

⁸Linearizable に複製するとは、すべてのレプリカが同じ順序でクライアント要求を処理し、その反映が書き込みリクエストのセッション内で必ず行われる性質である。各複製データにはシーケンス番号が割り当てられ、このシーケンス番号に従って処理される。

⁹チューリング完全な状態マシンでは、複製されるデータをオペレーション（状態遷移関数）と見なす。オペレーション f, g がある場合、一般に $f \circ g \neq g \circ f$ であるため、実行順序が重要になる。

PBFT のプロトコル詳細 (1)

PBFT の主なフェーズ：

1. Pre-prepare：プライマリ（リーダー） p がクライアント要求 r にシーケンス番号 s を割り当て、 $\langle \text{PRE-PREPARE}, s, r \rangle$ メッセージを他のレプリカに送信
2. Prepare：レプリカが要求の順序に合意し、 $\langle \text{PREPARE}, s, r \rangle$ メッセージを全レプリカに送信
3. Commit：少なくとも $2f + 1$ の PREPARE メッセージを受信したレプリカは $\langle \text{COMMIT}, s, r \rangle$ メッセージを送信し、実行を確約
4. Reply： $2f + 1$ の COMMIT メッセージを受信したレプリカは要求を実行し、結果をクライアントに返送

PBFT のプロトコル詳細 (2)

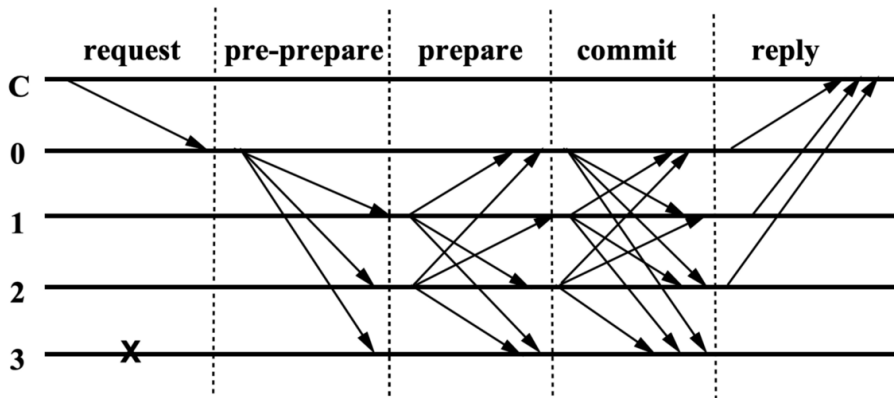


Figure 2: PBFT のシーケンシャルダイアグラム

PBFT の脅威モデル

PBFT の脅威モデルを考えてみよう。

PBFT の脅威モデル Λ_{PBFT}

- Corruption : **Byzantine** - レプリカは異なるメッセージを異なるレプリカに送信するなど、任意に振る舞うことができる
- Adaptivity : Adaptive - 敵対者はプロトコルの各フェーズで異なるレプリカを動的に腐敗させられる
- Computational Power : Polynomially Bounded - 多項式時間の計算に制限される
- Visibility : Private channels - 誠実なレプリカ間の通信内容は秘密であり、内部状態も敵対者からは見えない
- Mobility Fixed - 一度腐敗したレプリカはプロトコル実行中ずっと腐敗したままであり、回復することはない

PBFT の正当性保証

$n \geq 3f + 1$ のとき (f は腐敗ノードの数、 n はレプリカの総数) :

- ・ **安全性** (Safety) : 誠実なレプリカが同じシーケンス番号で異なる値をコミットすることはない
- ・ **活性** (Liveness) : すべてのクライアント要求は最終的に処理され応答が返される

を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

すなわち敵対者が脅威モデル Λ_{PBFT} の範囲においてどのような攻撃を行っても、Safety と Liveness を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

Question: なぜ $n \geq 3f + 1$ のとき Safety が満たされるのか？ PBFT がセキュアであることを説明してみよう！

仮定: 二つの異なる値 v と v' が同じシーケンス番号 s でコミットされた。

Lemma

各値がコミットされるためには、少なくとも $2f + 1$ 個のレプリカから同一の COMMIT メッセージが必要である。($3f + 1$ 個のレプリカのうち、腐敗したレプリカは最大 f 個)

- ・ 値 v がコミットされるには少なくとも $2f + 1$ 個のレプリカ群 $\mathcal{P}$ からの COMMIT メッセージが必要
- ・ 同様に、値 v' がコミットされるには少なくとも $2f + 1$ 個のレプリカ群 $\mathcal{Q}$ からの COMMIT メッセージが必要
- ・ $|\mathcal{P} \cap \mathcal{Q}| \geq 2f + 1 + 2f + 1 - n \geq 4f + 2 - (3f + 1) = f + 1$
- ・ $\mathcal{P} \cap \mathcal{Q}$
の中には少なくとも $f + 1$ 個のレプリカが存在し、システム全体で不正なレプリカは最大 f 個なので、
少なくとも 1 個の誠実なレプリカが両方の値 v と v' について COMMIT メッセージを送信したことになる
- ・ しかし、誠実なレプリカは同じシーケンス番号に対して異なる値の COMMIT メッセージを送信しないため、これは矛盾である
- ・ したがって、仮定は誤り

複雑化したプロトコルの正当性証明

実運用では PBFT のようなシンプルなプロトコルは稀であり、既存のプロトコルを部品として組み合わせた複合的なプロトコルが一般的である。

- ・ 既存のレプリケーション層（例：PBFT）上にアプリケーション層プロトコルを構築することが多い¹⁰

ハイブリッドプロトコル

n 個のプロトコル $\pi_1, \dots, \pi_n$ を組み合わせて得られるハイブリッドプロトコルを $\pi[\pi_1, \dots, \pi_n]$ と表記する。

Motivation：複合プロトコル $\pi[\pi_1, \dots, \pi_n]$ の正当性をどのように示すか？

- ・ 各構成要素を個別に分析して全体の正当性を証明できないか？
- ・ 任意の環境での正当性を保証するには？

¹⁰例：PBFT のうえにタイムスタンプ機構を作る. kekeho さんの GP2 に関連.

Universal Composability Framework

UC Framework を使えば、複雑なプロトコルの正当性を、シンプルな部品の完全性保証を元に示せる。つまり、複合プロトコル $\pi[\pi_1, \dots, \pi_n]$ が正当であることを証明するには、各構成要素 $\pi_1, \dots, \pi_n$ の正当性を証明し、それらの組み合わせ方が適切であることを示す。

理想機能 $\mathcal{F}$ の定義

理想機能 $\mathcal{F}$ とは、エンティティから入力を受け取り、 f を計算し、各エンティティに出力を返す。すなわち、プロトコルのインターフェースを定義し、任意の入力に対して、出力を返す写像 f を定義する。¹¹

プロトコル π の定義

プロトコル π とは、実際のプログラムであり、複数のエンティティ間で実行される手続き（プロトコルの具体的な実装）である。これは理想機能 $\mathcal{F}$ を実現するために設計され、メッセージの交換、計算処理、状態の管理などを含む具体的な手続きの集合である。

¹¹これは破壊不可能（すなわち、インターフェースや f の写像を変更することができない）な信頼されたエンティティとして考えることができる。

理想／現実世界パラダイム

UC には理想機能 $\mathcal{F}$ とプロトコル π を実行するために二つの世界がある:

理想世界 (Ideal World)

- ・ 理想機能 $\mathcal{F}$ が計算を行い、敵対者はシミュレータ Sim として振る舞う。
- ・ ダミーエンティティは入力を $\mathcal{F}$ に送り、 $\mathcal{F}$ からの出力を環境 $\mathcal{Z}$ に返す。
- ・ シミュレータ Sim は理想機能 $\mathcal{F}$ とのインターフェースを通じて、現実世界の敵対者の行動を模倣し、その結果を環境に返す。

現実世界 (Real World)

- ・ 現実世界は実プロトコル π とその参加エンティティ、敵対者 $\mathcal{A}$ で構成される。
- ・ エンティティたちはプロトコル π に従って互いに通信しながら計算を進める。
- ・ 敵対者 $\mathcal{A}$ は腐敗したエンティティを制御し、全ての内部状態やアクションが $\mathcal{A}$ によって制御される。

すなわち... それぞれの世界を以下のように解釈できる:

- ・ 理想世界は「完全に信頼できるエンティティ $\mathcal{F}$ が全ての処理を行う世界」と考えることができる。
- ・ 一方、現実世界は「信頼できるエンティティがないため、エンティティ同士が通信しながら処理を行う世界」と考えることができる。
- ・ したがって、プロトコルが正当であるとは、現実世界で各エンティティによって実行されたプロトコル π が、理想世界で実行された理想機能 $\mathcal{F}$ と区別できないことを意味する。^a

^a正確には無視できる確率で区別できないである。なぜなら、UC は確率的多項式時間 (PPT) 敵対者に対してのみ成立するからである。確率的多項式時間に関しては再掲となるが確率的チューリングマシンにおいてある問題の解法をオーダー $O(n^c)$ に近似して表記できる。

理想と現実世界を操作する環境エンティティ

この二つの世界を操作するエンティティが環境 $\mathcal{Z}$ である。

環境 $\mathcal{Z}$

- ・ 環境 $\mathcal{Z}$ はプロトコル外の全てのシステム（他のプロトコルなど）を抽象化したもの。
- ・ $\mathcal{Z}$ は理想世界ではダミーエンティティにプロトコル入力を送り、それらの出力とシミュレータ Sim からの出力を受け取る。¹²
- ・ $\mathcal{Z}$ は現実世界ではプロトコルを実行するエンティティに入力を送り、それらの出力と敵対者 $\mathcal{A}$ からの出力を受け取る。
- ・ $\mathcal{Z}$ は自分が相互作用している世界が理想世界か現実世界かを区別しようとする。¹³

¹² 正確には、環境は直接 $\mathcal{F}$ と通信せず、ダミーエンティティを介して間接的に相互作用する

¹³ つまり、 $\mathcal{Z}$ は現実世界における実プロトコルとエンティティの振る舞いと、理想世界における理想機能とダミーエンティティの振る舞いの違いを見つけようとする。

理想世界と現実世界の比較

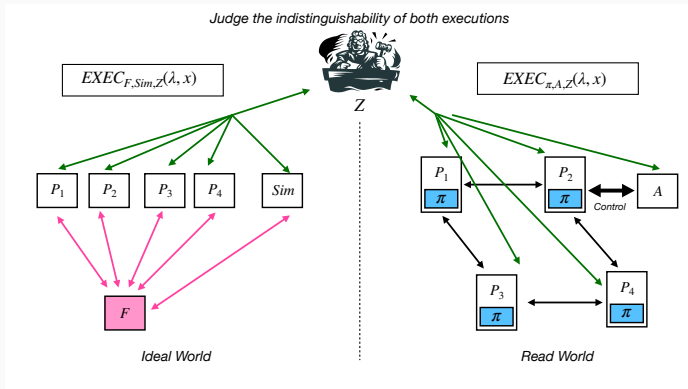


Figure 3: 理想世界と現実世界の相互作用. 理想世界では各ダミーパーティー P_i は理想機能 $\mathcal{F}$ と直接通信するが、現実世界ではプロトコル π を実行するエンティティ P_i 同士が通信する. このような見方をするので $\mathcal{F}$ は信頼する理想機能と考えることができる。

UCにおける正当性の主張

UC 正当性の定義

環境 $\mathcal{Z}$ は現実世界と理想世界の相互作用の違いを探そうとするが、どちらも区別できなければ π は理想機能 $\mathcal{F}$ を正当に実現していると言える。

これを“ π は $\mathcal{F}$ を UC-realize する”という。

厳密な定義. セキュリティパラメータ $\lambda \in \mathbb{N}$ とプロトコル π について、以下が成り立つとき、 π は理想機能 $\mathcal{F}$ を“UC-realize”するという：任意の確率的多項式時間¹⁴ 敵対者 $\mathcal{A}$ に対し，確率的多項式時間アルゴリズム Sim が存在し，任意の 確率的多項式時間環境 $\mathcal{Z}$ と任意の入力 $x \in \{0, 1\}^*$ について：

$$EXEC_{\mathcal{F}, Sim, \mathcal{Z}}(\lambda, x) \approx EXEC_{\pi, \mathcal{A}, \mathcal{Z}}(\lambda, x)$$

¹⁴UC では確率的多項式時間敵対者に対してのみ成立する

UC を使うと、正当性証明において何が嬉しいのか？

- ・ モジュール性 (Modularity). 複雑な課題を小さなプロトコルに分解でき、各プロトコルの正当性を独立して証明できる。
- ・ 合成定理 (Composition Theorem). プロトコル π が理想機能 $\mathcal{F}$ を UC-realize し、プロトコル ρ が $\mathcal{F}$ を使って理想機能 $\mathcal{G}$ を UC-realize するなら、 $\mathcal{F}$ を π に置き換えたプロトコル $\rho^{\mathcal{F} \rightarrow \pi}$ も $\mathcal{G}$ を UC-realize する。

UC フレームワークの核心は「合成可能性 (Composability)」：一度証明されたプロトコルの正当性は、どのような複雑な環境でも保持される。これにより、安全なプロトコルの段階的な構築と再利用が可能になる。

プロトコルインスタンスとセッション識別

実世界では、同じプロトコルの複数のコピーが異なるユーザーによって同時に実行される。プロトコルはいわば型であり、実際にはその型の実体であるインスタンスが実行される。

プロトコルインスタンス

プロトコル π のインスタンスとは、そのプロトコルの具体的な実行例であり、通常は一意的識別子で区別される。

- ・ セッション識別子 (*sid*)：各プロトコル実行を識別する一意の識別子
- ・ たとえば... ブロックチェーン上では、スマートコントラクトのアドレスが *sid* として機能する

基本的な UC フレームワークでは、各プロトコルインスタンスは独自のインスタンスを持つ。しかし、これは現実的ではない場合がある（例：同じハッシュ関数が複数のプロトコルで使用される場合）。

Generalized Universal Composability (GUC) Framework

GUC Framework とは

UC の拡張フレームワークで、複数のプロトコルインスタンスで共有される“グローバル機能”の概念を導入している。

グローバル機能 (Global Functionality)

- ・ 複数のプロトコルインスタンスで共有される機能
- ・ 例：ハッシュ関数、デジタル署名、共有台帳など
- ・ すべてのプロトコルセッションで一貫した内部状態を維持

GUC によって、並列実行環境を実現するだけでなく、インスタンスを個別に分析する必要がない。

なぜこのような正当性証明方法を身につける必要がある？

学術に受け入れられるフォーマットで学術論文を書く。^aじゃないとその時点でプロトコルの設計一見が良くても門前払いされる。

^aとりわけ国際ジャーナルやトップコンファレンスと呼ばれているところは。トップコンファレンスとは採択率は 20%前半はそれ以下、もしくは CORE ランクが A や A⁺ のコンファレンス

ただ、プロトコルの仕様や実装を書くだけでは

“ Since there is no formal security model, it is hard to verify the formal correctness of the claims. Moreover, the scheme includes many low-level implementation details instead of giving a general description of their scheme. ” と言われる。

(moz が 2 年前に CRYPTO'23 というトップコンファレンスでもらった査読コメント抜粋)



International Association for Cryptologic Research (IACR)

<https://eprint.iacr.org> > ... PDF

頻繁にアクセス

A New Paradigm for Cryptographic Protocols

R Canetti 著・2000・被引用数: 4519 — **Composability** (or, rather, **de-composability**) can be regained via using either the **Universal**. Composition with Joint State (JUC) theorem or ...

トップコンファレンスに出版されてる論文を眺めてみよう

“FairSwap: How to Fairly Exchange Digital Goods” という ACM CCS 2020 というトップコンファレンスで出版された論文を眺めてみよう。

この論文は、 $\mathcal{A}$ と $\mathcal{B}$ の間でデジタルデータ x と y を交換するプロトコルを提案している。 $P: \mathcal{A} \xrightarrow{x} \mathcal{B}, Q: \mathcal{B} \xrightarrow{y} \mathcal{A}$ をどちらも満たすか、どちらも満たさないかを保証するプロトコル

この理想機能 $\mathcal{F}_{fe}$ を定義した上で、ブロックチェーン機能をもつグローバル機能 $\mathcal{L}$ とランダムオラクル $\mathcal{H}$ を定義している。

<https://dl.acm.org/doi/pdf/10.1145/3243734.3243857>



FairSwap: How to Fairly Exchange Digital Goods

Stefan Dziembowski
Institute of Informatics
University of Warsaw, Poland
stefan.dziembowski@crypto.edu.pl

Lisa Ekey
Department of Computer Science
TU Darmstadt, Germany
lisa.ekey@cs.tu-darmstadt.de

Sebastian Faust
Department of Computer Science
TU Darmstadt, Germany
sebastian.faust@cs.tu-darmstadt.de

ABSTRACT

We introduce FairSwap – an efficient protocol for fair exchange

Unfortunately, it has been shown that without further assumptions it is impossible to design protocols that guarantee such strong fairness properties [90]. To overcome this impossibility

本当に仮定はこれだけでいいのか？

ここまでで用いた仮定は以下の通りである：

- ・ ネットワークモデル：非同期か同期か、部分同期（通信遅延に上限がある）か
- ・ 敵対者モデル：敵対者はどのように振る舞うか
- ・ 腐敗ノードの数：攻撃者が制御できるエンティティの数の条件 ($n \geq 3f + 1$ など)

Question: 実世界で動かす上で考えるべき脅威、仮定はこれだけか？

これだけではすべてのプロトコルに Sybil Attack が適用できてしまう。

Sybil Attack

単一のエンティティが複数のエンティティを模倣し、システム内で不当な影響力を得る攻撃である。¹⁵

PBFT では $n \geq 3f + 1$ の条件がある。 n はノードの数、 f は敵対者の数である。これは各レプリカが一意に識別可能であり、メッセージの送信元に偽りがないことを暗黙の前提においている敵対者が単一のエンティティ（敵対者）が複数のエンティティとして振る舞えることを許すと、条件が破綻する。^a

^aCorruption Model の Byzantine に一定の妥協を許している。これはいいのか？

¹⁵ 1 人のエンティティが山田花子と山田次郎の二つの役を演じているが、他のエンティティは山田花子と山田次郎が同一のエンティティであるかどうかを判断できない。

デジタル署名

メッセージの送信元に偽りがないことを検証するためには、メッセージにデジタル署名を付与し、デジタル署名を検証する必要がある。

デジタル署名

数学的な関係性がある公開鍵 (verification key) pk と秘密鍵 (signing key) sk のペアを用いて、メッセージの送信元を検証する。

- ・ メッセージ m をデジタル署名する： $\sigma \leftarrow \text{SIGN}(sk, m)$
- ・ メッセージ m と署名 σ を検証する： $\{0, 1\} \leftarrow \text{VERIFY}(pk, m, \sigma)$ where 1 if m is valid, 0 otherwise

具体的なデジタル署名スキームとしては、RSA、ECDSA、EdDSA、Schnorr 署名などがある。

メッセージの真正性の検証

エンティティ $\mathcal{P}$ がメッセージ m をエンティティ $\mathcal{Q}$ に送付したとする。エンティティ $\mathcal{Q}$ はメッセージ m を確かに $\mathcal{P}$ から受け取ったことを検証する必要がある。

- ・ エンティティ $\mathcal{P}$ がメッセージ m に対するデジタル署名 σ を公開鍵 $pk_{\mathcal{P}}$ に対応する秘密鍵 $sk_{\mathcal{P}}$ を用いて生成する。
- ・ $\mathcal{P}$ は (m, σ) を $\mathcal{Q}$ に送付する。
- ・ $\mathcal{Q}$ は受け取った (m, σ) を検証する： $\text{VERIFY}(pk_{\mathcal{P}}, m, \sigma) \stackrel{?}{=} 1$

検証の正当性確保に必要な仮定: $\mathcal{Q}$ は $\mathcal{P}$ の公開鍵 $pk_{\mathcal{P}}$ を知っている。

Question: これだけで Sybil Attack を防げるか？

識別子の生成コスト

Answer: 不十分。“デジタル署名で送信元を区別できる $\Leftrightarrow$ エンティティを区別できる”という命題は必ずしも成立しないからである。

デジタル署名はメッセージの送信者を区別できるが、エンティティと公開鍵の一対一対応が仮定されなければ、検証の正当性は保証されない。

- ・ 単一のエンティティが複数の公開鍵を生成し複数のエンティティとして振る舞える
- ・ エンティティと公開鍵の一対一対応を保証するためには Key Management System¹⁶が必要。KMS は一対一対応の検証する¹⁷

¹⁶Key Management System (KMS) は、エンティティと公開鍵の一対一対応を保証するシステムである。Public Key Infrastructure (PKI) は Key Management System の一種である。

¹⁷検証は二つある。第一にあるエンティティが自分は山田花子と主張したが、本当に山田花子であるかどうかを検証する。第二にエンティティ群と公開鍵の写像を定義すると、その関係性が全単射であることを確認する。確認後、KMS は紐付きが有効であることを示す署名 (=証明書) を発行する。利用者は KMS を信頼することで、この検証を署名の検証に置き換えることができる。

しかし、KMS には以下の問題がある：

- ・ 認証局の存在を許す = 信頼できる第三者の存在を仮定する
- ・ 単一障害点とスケーラビリティの問題が生じる

ほかにも Proof-of-Work：新たな識別子の作成に計算資源を要求する研究もある。

- ・ 分散システムの文脈で (Byzantine Fault Tolerance な) プロトコルを俯瞰すると、分散システムの魅力や効力を最大限発揮できないボトルネック（暗黙、非自明な仮定）が存在する。それは必ずしも学術論文に書かれているとは限らない。

どうやったらインパクトのある問題を見つけるか？

- ・ 運用を片手に持つ、ということは、理論の中で議論されたプロトコルを解釈する選択肢を広げることと等しい。
- ・ すなわちそのプロトコルが置かれる状況（提案手法の論理を支える仮定^a）を運用から得られた洞察をもとに変えてみて、ボトルネックが無視できるか、できないかを判断できる。
- ・ そのような探求がインパクトがあり、明確な Problem Statement に役立つと moz は信じている。

^a例えば、敵対者モデル、ネットワーク、タイミングモデル

今回は理論的な視点を提供したが、ぜひ運用視点を他の kG から聞くことで、自身の研究（分野）に新たな解釈をもたらすかもしれない。そのような解釈を皆で有機的に生み出し、共有し、高め合う全体講義になればと願っている。